

FishErLes Size and functional TYpe model (FEISTY) in R

Yixin Zhao, P. Daniël van Denderen, Karline Soetaert, and Ken H. Andersen

March, 2025

Introduction

This vignette provides a comprehensive description of the model equations, parameters, and numerical implementation used in the FishErLes Size and functional TYpe (FEISTY) model system (Petrik et al. 2019; Denderen et al. 2021). This text is followed by several examples illustrating how to use the model within R.

The FEISTY model

The FEISTY model framework is designed to be mechanistic, simple and fast, generally applicable globally, and compatible with biogeochemical model principles. It is based on ordinary differential equations and careful accounting of mass balancing. The model structures fish around functional types (aka. functional groups or guilds) based on traits instead of representing specific species. This simplification, together with the mechanistic basis, allows for projections into novel environments, e.g., climate change projections.

The current FEISTY implementation includes four setups that define which fish functional types are present and how they predate upon one another. Two “Basic” setups contain three functional types: small and large pelagic fish species and demersal species (Petrik et al. 2019). Two “Vertical” setups augment the model with two additional functional types, mesopelagic fish and midwater predators, and further automatize the calculation of the interaction between the functional types based on their vertical overlap (Denderen et al. 2021). The Basic and Vertical setups with the suffix “2” modify the published setups with minor parameter and structural changes. All setups are detailed below.

Regardless of the setup type the biomass dynamics follow the same set of equations, which is detailed in the next section. The differences between the setups lie in how the interactions between the types and sizes are determined, which is detailed in the section “Predation”.

Biomass dynamics

Fish dynamics

FEISTY is a size-based model where the biomass of each size class of a functional type of fish (B_i , gWW m^{-2}) ¹ changes over time (t) according to source and sink of energy determined by consumption, growth, reproduction and mortality (De Roos et al. 2008). The changes in biomass are described by:

$$\frac{dB_i}{dt} = J_{\text{in},i} + (\nu_i - \rho_i - \mu_i)B_i - J_{\text{out},i}, \quad (1)$$

¹gWW: gram wet weight. From now on, we use “g” to represent “gWW” for simplicity.

where ν_i is the rate of available energy for growth and reproduction, ρ_i is the rate of reproduction, and μ_i is the total mortality rate. Fish growth leads to a flux out of each size class, which enters the next size class. Since the largest size-class of a functional type cannot grow to an even larger size, the flux out of the last size-class is used for reproduction. The biomass flux out of a size class i is:

$$J_{\text{out},i} = \gamma_i B_i, \quad (2)$$

where γ_i is the growth rate (year^{-1}). The growth flux into a size class differs between the smallest size-class of each functional type and the other sizes. These two parts are:

$$J_{\text{in},i} = \begin{cases} \epsilon_r \left(J_{\text{out},n} + \sum_i \rho_i B_i \right), & i = 1 \\ J_{\text{out},i-1}, & i > 1 \end{cases} \quad (3)$$

where ϵ_r is the reproduction efficiency. The flux into the smallest size class ($i = 1$) is from the reproduction of each size class of mature fish plus the biomass flux out of the maximum size class ($i = n$) of a functional type ($J_{\text{out},n}$). For all other size classes ($i > 1$), the growth flux into the size class i is from the growth of the previous neighboring size class ($i - 1$).

Resource dynamics

FEISTY coupling with the biophysical forcing can be accomplished in three ways: fully two-way (online), production-constrained one-way (offline), and with resources as a semi-chemostat (offline). In case of the latter, the resource i biomass dynamics change over time according to a chemostat-like growth and consumption by fish predators. The modelling of resources as a semi-chemostat is the default resource dynamic mode in the four internal setups:

$$\frac{dR_i}{dt} = r(K_i - R_i) - \mu_{p,i} R_i, \quad (4)$$

where R is the state variable of a resource, K is the carrying capacity (g m^{-2}), and r is the growth rate which is a constant value (1 year^{-1}). μ_p represents the predation mortality on the resource. Lastly, FEISTY has an option to model resource dynamics with the logistic growth equation:

$$\frac{dR_i}{dt} = r_i \cdot R_i \cdot \left(1 - \frac{R_i}{K_i}\right) - \mu_{p,i} \cdot R_i, \quad (5)$$

which can be set in `paramAddResource()` function.

Physiological rates

There are three main biological rates based on the allometric scaling relationship of individual body size: the mass-specific maximum consumption rate $C_{\text{max},i}$ (year^{-1}), the mass-specific clearance rate V_i ($\text{m}^2 \text{ g}^{-1} \text{ year}^{-1}$), and the mass-specific metabolic rate M_i (year^{-1}):

$$C_{\text{max},i} = k_{T,i} \cdot a_c \cdot m_i^{b_c} \quad (6)$$

$$V_i = k_{T,i} \cdot a_e \cdot m_i^{b_e} \quad (7)$$

$$M_i = k_{TM,i} \cdot a_m \cdot m_i^{b_m} \quad (8)$$

Here $k_{T,i}$ and $k_{TM,i}$ are scaling factors of temperature effects of each size class (see Section **Temperature effects**). a_c , a_e , and a_m are factors for each biological rate; b_c , b_e , and b_m are exponents. These biological rates determine the energy gained from predation and costs due to basal metabolism (see Section **Energy budget**).

In the code implementation, these basic biological rates of fish (set against the reference temperature of 10°C) are assigned by the `paramAddPhysiology()` function. The scaling of biological rates with temperature is added subsequently (see Section **Temperature effects**).

Energy budget

For a given individual size of fish m_i , the mass-specific available energy for growth or reproduction (rate) ν_i is the result of the mass-specific energy from assimilated food (rate) minus the mass-specific metabolic rate:

$$\nu_i = \epsilon_\alpha f_i C_{\max,i} - M_i \quad (9)$$

where ϵ_α is the assimilation efficiency. The mass-specific feeding level f_i (dimensionless) describes how much food a predator can eat relative to the maximum consumption capability, ranging from 0 to 1:

$$f_i = \frac{E_i}{E_i + C_{\max,i}} \quad (10)$$

Therefore, the mass-specific consumption rate $f_i C_{\max,i}$ is based on type II functional response. The feeding behavior of a predator is a consequence of encountering prey. The mass-specific encounter rate (year⁻¹) E_i of a predator i is:

$$E_i = V_i \sum_j \theta_{i,j} B_j, \quad (11)$$

where $\theta_{i,j}$ denotes the feeding preference of a predator i on a prey j (see Section **Predation**). B_j represents the biomass of a prey j including zooplankton, benthos, or fish. Therefore, the $\sum_j \theta_{i,j} B_j$ represents the biomass of all prey available to a predator.

Reproduction and growth

When the assimilated energy is positive after meeting the metabolic demands, energy is available for fish growth and reproduction (when the available energy is negative, there will be a biomass loss through starvation). The reproductive rate (year⁻¹) of each size class of fish is:

$$\rho_i = \begin{cases} \psi_i \nu_i, & \nu_i > 0 \\ 0, & \nu_i \leq 0 \end{cases} \quad (12)$$

ψ_i is the maturity level (dimensionless) of fish i , describing the proportion of available energy (ν_i) used in the reproduction process. The maturity level can either be assigned manually (setupBasic and setupVertical) or by a sigmoid function (setupBasic2 and setupVertical2):

$$\psi_i = \left(1 + \left(\frac{m_{c,i}}{m_{\text{mature}}} \right)^{-5} \right)^{-1} \quad (13)$$

where m_{mature} is the half-maturation size of a functional type, which means fish reach 50% maturity level at m_{mature} .

$$m_{\text{mature}} = \eta_{\text{mature}} M \quad (14)$$

where η_{mature} is the coefficient determining the half-maturation size relative to the maximum size (boundary size) of a functional type (M).

κ_i represents the fraction of the available energy (ν_i) invested in growth:

$$\kappa_i = 1 - \psi_i \quad (15)$$

The available energy for growth $\kappa_i \nu_i$ gives a flux of biomass out of the size-class i that depends on the ratio z between the smallest and largest size of the size-class and total mortality μ_i (De Roos et al. 2008):

$$\gamma_i = \begin{cases} \frac{\kappa_i \nu_i - \mu_i}{1 - (1/z)^{1 - \mu_i / \kappa_i \nu_i}}, & \nu_i > 0 \\ 0, & \nu_i \leq 0 \end{cases} \quad (16)$$

μ_i includes a predation ($\mu_{p,i}$), background ($\mu_{b,i}$), and fishing mortality rate ($\mu_{f,i}$):

$$\mu_i = \mu_{p,i} + \mu_{b,i} + \mu_{f,i} \quad (17)$$

All size classes of each functional type experience a background mortality. The details of predation and fishing are described in the sections below.

Predation

FEISTY specifies of how each size class (and zooplankton and benthic prey resources) interact via predator-prey interactions in a feeding preference matrix (also known as a food-web matrix). In this matrix, each element $\theta_{i,j}$ is a number between 0 and 1, which denotes how much a fish class i preys on a prey class j . The feeding preference function is either based on size preference (see Section **Size preference**) or derived from size preference coupling with the vertical habitat of each functional group (see Section **Vertical overlap**). For some combinations of predator and prey, additional modifications are included to account for feeding specializations (see Section **Modifications to the preference matrix**).

Once the preference matrix is established, the predation mortality of a prey j can be calculated:

$$\mu_{p,j} = \sum_i \frac{V_i \theta_{i,j}}{E_i + C_{\max,i}} C_{\max,i} B_i. \quad (18)$$

This describes the rate at which a prey j (either a resource or a fish size class) is eaten by all its predators (in nunit of per time). When j denotes a resource, $\mu_{p,j} R_j$ is the predation loss of the resource (the second term in Eq. 4).

Size preference

In the non-vertical setups (setupBasic and setupBasic2) $\theta_{i,j}$ refers to the feeding preference based on the size preference. In setupVertical and setupVertical2, it is named $\theta_{s,i,j}$ for differentiation purposes. The value of $\theta_{i,j}$ can be assigned manually (setupBasic, see Table 2 in (Petrik et al. 2019)) or it can be calculated using size-preference functions that are either based on the log-normal distribution (setupBasic2 and setupVertical2):

$$\theta_{i,j} = \begin{cases} \exp \left(-\frac{\left(\log \left(\frac{m_{c,i}}{\beta \cdot m_{c,j}} \right) \right)^2}{(2 \cdot \sigma)^2} \right), & m_i > m_j \\ 0, & m_i \leq m_j \end{cases} \quad (19)$$

or the error function (setupVertical, Denderen et al. 2021):

$$\theta_{i,j} = \frac{\sqrt{\frac{\pi}{2}} \cdot \sigma \cdot \left(\operatorname{erf} \left(\frac{\log(m_{u,j}) - \log(\frac{m_{c,i}}{\beta})}{\sqrt{2} \cdot \sigma} \right) - \operatorname{erf} \left(\frac{\log(m_{l,j}) - \log(\frac{m_{c,i}}{\beta})}{\sqrt{2} \cdot \sigma} \right) \right)}{\log(m_{u,j}) - \log(m_{l,j})} \quad (20)$$

where β is preferred predator-prey mass ratio, and σ is the size preference width for feeding. The size-preference function selection and the values of β and σ can be defined by the function `paramSizepref()`.

Vertical overlap

The vertical distribution in setupVertical and setupVertical2 is modeled in a discretized water column:

$$z_\zeta = 0 + \zeta, \quad \zeta \in [0, z_b] \quad (21)$$

where ζ is each depth of a water column (integer value), ranging from 0 (surface) to z_{bottom} (sea floor).

The vertical distribution of each size class and functional type i is described as a unimodal distribution:

$$\theta_{\zeta,x,i} = \frac{1}{\sqrt{2\pi\omega_i^2}} \cdot \exp\left(-\frac{(z_{\zeta} - z_{\text{loc},x,i})^2}{2\omega_i^2}\right) \quad (22)$$

where ω_i gives the vertical range of the vertical distribution of each size class and functional type, x is either day or night, and $z_{\text{loc},x,i}$ is the defined water column depth with the maximum biomass concentration. Some size classes in some functional types have two depths with the maximum biomass concentration $z_{\text{loc},x,i}$. Consequently, they need to be calculated twice and the vertical distribution is the averaged value, which shows a bimodal shape of the vertical distribution. The final vertical distribution in either day or night at each depth of the water column ζ is re-scaled so that the sum over the water column is equal to one:

$$\theta_{\zeta,x,i} = \frac{\theta_{\zeta,x,i}}{\sum_{\zeta} \theta_{\zeta,x,i}} \quad (23)$$

The vertical range of the vertical distribution of each organism ω_i increases with body mass as larger individuals typically have a wider habitat range:

$$\omega_i = \omega_0 + \tau \cdot \log_{10}\left(\frac{m_{c,i}}{m_{c,0}}\right) \quad (24)$$

where ω_0 is the baseline range of the vertical distribution and τ is the range-increasing factor of vertical distribution. $m_{c,0}$ denotes the reference size to which all organisms are scaled in the simulation (typically small mesozooplankton).

Vertical overlap between predator and prey is then estimated as the lowest value between vertical distribution values of a specific predator i and a prey j at depth ζ at daytime or nighttime x $\min(\theta_{\zeta,x,i}, \theta_{\zeta,x,j})$. The depth-integration gives the synthetic vertical overlap value of predator i to a prey j at day or night $\theta_{v,x,i,j}$:

$$\theta_{v,x,i,j} = \sum_{\zeta} \min(\theta_{\zeta,x,i}, \theta_{\zeta,x,j}) \quad (25)$$

Ultimately, the total vertical overlap is the averaged value of the synthetic day and night vertical overlap values:

$$\theta_{v,i,j} = \frac{\theta_{v,\text{day},i,j} + \theta_{v,\text{night},i,j}}{2} \quad (26)$$

This depth integrated vertical overlap value $\theta_{v,i,j}$ is combined with the size-based preference $\theta_{s,i,j}$ and used for the calculation of feeding preference $\theta_{i,j}$:

$$\theta_{i,j} = \theta_{s,i,j} \cdot \theta_{v,i,j} \quad (27)$$

Vertical habitats of resources and fish

Regardless of water conditions, Benthos are concentrated at the bottom ($z_{\text{loc},\text{day},i} = z_{\text{loc},\text{night},i} = z_{\text{bottom}}$) and to ensure they remain closely associated with the sea floor, their vertical range is kept as the baseline value $\omega_i = \omega_0$ (used in Eq. 22).

The vertical habitat strategy of fish depends on seafloor depth and varies between fish functional types. Small and large pelagics and demersal fish make up the fish community in shelf regions, i.e. regions < 250 meters in depth. In these regions, zooplankton prey, small and large pelagics are distributed in the upper water column

with a maximum concentration at the surface of the water column ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = 0$). Demersal fish change their vertical distribution throughout ontogeny ²: small-size classes feed in the upper water column ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = 0$), medium-size classes feed at the bottom ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = z_{\text{bottom}}$), and large-size classes, if the water column is full of light ($z_{\text{bottom}} \leq z_{\text{photic}}$), exhibit cross-habitat feeding between the bottom and the surface waters all the day with maximum concentrations at the surface and the bottom (two $z_{\text{loc},x,i}$ for each day and night: 0 and z_{bottom}). If ($z_{\text{bottom}} > z_{\text{photic}}$) large demersal fish stay at surface waters during daytime ($z_{\text{loc,day},i} = 0$) and bottom at night ($z_{\text{loc,night},i} = z_{\text{bottom}}$).

In slope and open ocean regions, some zooplankton resources and size classes of specific fish functional types conduct diel vertical migration to the depth z_{dvm} ($z_{\text{dvm,dem}}$ for large demersal fish, described later), which is the representative depth of maximum biomass concentration:

$$z_{\text{dvm}} = \begin{cases} z_{\text{photic}} + 500, & z_{\text{bottom}} \geq z_{\text{photic}} + 500 \\ z_{\text{bottom}}, & z_{\text{photic}} + 500 > z_{\text{bottom}} > z_{\text{shelf}} \\ 0, & z_{\text{bottom}} \leq z_{\text{shelf}} \end{cases} \quad (28)$$

The migrating part of the zooplankton community is at depth during the day and at the surface at night, whereas the remaining zooplankton stay at the surface waters the whole day ($z_{\text{loc,day},i} = 0$ and z_{dvm} ; $z_{\text{loc,night},i} = 0$). As a result, during the day, zooplankton have a bimodal distribution with maximum concentration depths at the surface and in the twilight zone.

Small pelagics are distributed in the upper water column with a maximum concentration at the surface water ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = 0$).

Mesopelagic fish follow the day/night vertical movement of the migrating zooplankton with a maximum concentration at similar depths ($z_{\text{loc,day},i} = z_{\text{dvm}}$; $z_{\text{loc,night},i} = 0$).

Large pelagics, perform differently throughout ontogeny. The small- and medium-size classes are distributed in the upper water column with a maximum concentration at the surface at day and night ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = 0$). The large size-classes have a bimodal distribution during the day, where half are distributed in the upper water column with a maximum concentration at the surface ($z_{\text{loc,day},i} = 0$), and the other half have a maximum concentration in the midwater ($z_{\text{loc,night},i} = z_{\text{dvm}}$). At night, they are distributed in the upper water column ($z_{\text{loc,night},i} = 0$).

Small- and medium-size classes of midwater predators have the same vertical distribution as mesopelagic fish ($z_{\text{loc,day},i} = z_{\text{dvm}}$; $z_{\text{loc,night},i} = 0$). Large midwater predators have a maximum concentration in midwater ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = z_{\text{dvm}}$).

Small demersals are distributed in the upper water column with a maximum concentration at the surface both day and night ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = 0$). Medium demersal fish are distributed near the bottom ($z_{\text{loc,day},i} = z_{\text{loc,night},i} = z_{\text{bottom}}$). Large demersal fish have a vertical habitat strategy that differs from the other groups. At night, large demersal fish are at the bottom ($z_{\text{loc,night},i} = z_{\text{bottom}}$), while during the day, the depth of maximum concentration $z_{\text{dvm,dem}}$ is:

$$z_{\text{dvm,dem}} = \begin{cases} z_{\text{dvm}}, & z_{\text{bottom}} - z_{\text{dvm}} < 1200 \\ z_{\text{bottom}} - 1200, & 1200 \leq z_{\text{bottom}} - z_{\text{dvm}} < 1500 \\ z_{\text{bottom}}, & z_{\text{bottom}} - z_{\text{dvm}} \geq 1500 \end{cases} \quad (29)$$

Table 1 showed a summary of vertical habitats for different resources and functional types.

²small-, medium-, and large-size classes of fish are distinguished by comparison between the geometric mean mass value ($m_{c,i}$) and boundary mass value (m_{medium} or m_{large}). Small-size class: $m_{c,i} < m_{\text{medium}}$. Medium-size class: $m_{\text{medium}} \leq m_{c,i} \leq m_{\text{large}}$. Large-size class: $m_{c,i} > m_{\text{large}}$. Default $m_{\text{medium}} = 0.5\text{g}$ and $m_{\text{large}} = 250\text{g}$.

Table 1: Representative depth of maximum biomass of each size class $z_{loc,x,i}$ at day and night. z_{dvm} and $z_{dvm\text{dem}}$ can be found in Eq. 28 and 29.

	Day			Night		
Resources						
Zooplankton ^a	0 and z_{dvm}			0		
Benthos	z_{bottom}			z_{bottom}		
Fish	Small	Medium	Large	Small	Medium	Large
Small pelagic fish	0	0	/	0	0	/
Mesopelagic fish	z_{dvm}	z_{dvm}	/	0	0	/
Large pelagic fish	0	0	0 and z_{dvm}	0	0	0
Midwater predators	z_{dvm}	z_{dvm}	z_{dvm}	0	0	z_{dvm}
Demersal fish	0	z_{bottom}	$z_{dvm\text{dem}}^b$	0	z_{bottom}	z_{bottom}^b

^a Zooplankton include small and large mesozooplankton

^b If the water column is shallower than the photic zone depth ($z_{\text{bottom}} \leq z_{\text{photic}}$), large demersal fish migrate over the water column (two $z_{loc,x,i}$ for each day and night: $z_{dvm\text{dem}}$ and z_{bottom}). In such cases, $z_{dvm\text{dem}} = 0$.

Modifications to the preference matrix

setupBasic2

- The feeding preference from small pelagic and large pelagic fish to benthic resources and medium demersal fish are set to 0.
- Large pelagic fish have a reduced feeding preference for medium-sized small pelagic fish ($\theta_A \cdot \theta_{i,j}$).
- The feeding preference from small demersal fish to benthic resources is set to 0.
- Medium demersal fish only feed on benthos and themselves (cannibalism). Therefore, their feeding preference on zooplankton and all fish excluding themselves is corrected to 0.
- Large demersal fish eat both pelagic and benthic organisms in shallow water (< 200 m). They are less efficient at attacking pelagic prey and hence their feeding preference on small pelagic fish is down-regulated: $\theta_A \cdot \theta_D \cdot \theta_{i,j}$; similarly for their feeding preference on large pelagic fish: $\theta_D \cdot \theta_{i,j}$. In deeper waters (> 200 m), large demersal fish only feed on benthos, medium demersals, and themselves.

setupVertical and setupVertical2

- Small and large pelagics are visual predators whose predation ability is better in light-rich waters during the day and worse in dark conditions such as night or the twilight zone. Therefore, according to habitats (Table 1), the vertical overlap is modified by multiplying it with a visual scaling factor θ_{visual} :

$$\theta_{v,x,i,j} = \theta_{v,x,i,j} \cdot \theta_{\text{visual}}. \quad (30)$$

During daytime, the vertical overlap of all small and large pelagics on all preys in surface waters ($\theta_{v,\text{day},i,j}$) is enhanced ($\theta_{\text{visual}} = 1.5$); the vertical overlap ($\theta_{v,\text{day},i,j}$) of large size classes of large pelagics on the preys in the twilight zone (z_{dvm}) (mesopelagic fish and midwater predators) is reduced ($\theta_{\text{visual}} = 0.5$). At night, their vertical overlap on all preys ($\theta_{v,\text{night},i,j}$) is decreased ($\theta_{\text{visual}} = 0.5$). Note this modification is done before Eq. 26.

- The feeding preference from all size-classes that are pelagic-living (small pelagics, mesopelagics, large pelagics, midwater predators and small size classes of demersal fish) to benthic resources are set to 0. In addition, their feeding preference on medium demersal fish (benthic-living) is reduced ($\theta_{i,j} = \theta_{i,j} \cdot 0.25$).
- The feeding preference of medium and large demersal fish to zooplankton resources are set to 0.

- All large-size classes of pelagic-living functional fish groups (large pelagic fish, midwater predators, and demersal fish) have reduced feeding preference on medium-size small pelagic fish and medium-size mesopelagic fish ($\theta_{i,j} = \theta_{i,j} \cdot \theta_A$).

Fishing mortality

The fishing mortality rate $\mu_{f,i}$ of a particular size class of fish i can be either determined by constant values or by a fishing selectivity function (Andersen 2019, chap. 5):

$$\mu_{f,i} = \psi_{f,i} F_{\max} \quad (31)$$

where $F_{\max}$ is the baseline fishing mortality rate (year^{-1}) of a functional type of fish. $\psi_{f,i}$ denotes a trawl-based fishing selectivity following a sigmoid function:

$$\psi_{f,i} = \left(1 + \left(\frac{m_{c,i}}{m_{\text{fishing}}} \right)^{-3} \right)^{-1} \quad (32)$$

where m_{fishing} indicates the fish with this size is under 50% harvesting rate.

$$m_{\text{fishing}} = \eta_{\text{fishing}} M \quad (33)$$

where η_{fishing} controls the weight of fish with a 50% harvesting rate relative to the max weight of the functional type. The fishing mortality rate can be assigned by the function `setFishing()`.

Temperature effects

The temperature effects on the mass-specific maximum consumption rate $C_{\max,i}$, the mass-specific clearance rate V_i , and the mass-specific metabolic rate M_i (Eq. 6, 7, 8) are based on the use of the Q_{10} coefficient, which describes the exponential variation of rates every 10°C .

The general equation for the temperature scaling factor k is:

$$k = Q_{10}^{\frac{T - T_{\text{ref}}}{10}} \quad (34)$$

where T_{ref} is the reference temperature and T is the environment temperature. According to the different functional types of fish and their sizes, fish habitats change. For fish staying in the pelagic zone, $T = T_p$, which reflects the average temperature in the top 100m of the water column. For fish in the benthic zone, $T = T_b$, which reflects the temperature near the bottom. Through introducing different Q_{10} coefficients (Table 2), various temperature scaling factors k can be obtained. k_T is the temperature scaling factor for $C_{\max,i}$, and V_i ; k_{TM} is the temperature scaling factor for M_i .

setupBasic and setupBasic2

In `setupBasic` and `setupBasic2`, there are three functional types of fish: small pelagic fish, large pelagic fish, and demersal fish. Small pelagics and large pelagics are always in the pelagic water, so the T_p is consistently applied. Demersal fish habitats change according to size. Small demersal fish are in a pelagic status (T_p). Medium demersals live at the bottom (T_b). Large demersals also stay in the bottom if the water is deep (> 200 m). However, in shallow water (< 200 m), large demersals are assumed to spend a fraction of time λ in the pelagic zone and the rest of time $1 - \lambda$ at the bottom according to the abundance of prey in these two zones:

$$\lambda = \frac{B_{\text{pelprey}}}{B_{\text{allprey}}} \quad (35)$$

where B_{pelprey} denotes the total biomass of pelagic prey for large demersal fish, B_{allprey} is the total biomass of all prey for large demersal fish. A simple case can be found in Eq. 15 in (Petrik et al. 2019).

Therefore, the temperature for large demersals is defined based on their time spent in the pelagic zone and benthic zone, which is called effective temperature:

$$T_e = T_p \cdot \lambda + T_b \cdot (1 - \lambda) \quad (36)$$

In code implementation, temperature effects on biological rates are done by the function `paramTeffect()`. Note this function only works on non-vertical distribution setups, i.e., `setupBasic` and `setupBasic2`. T_e is updated each time step during the time integration, along with the temperature-dependent biological rates of large demersals, which are handled by the function `updateET()`. The function `updateET()` is called every time step in `derivativesFEISTYR()`. The effective temperature scheme is forced turned on in `setupBasic()` (Petrik et al. 2019); it is an option in `setupBasic2()`.

setupVertical

In `setupVertical`, fish have a vertical distribution. Temperature effects on physiological rates of each size class are based on where fish stay in a water column (environmental temperature). Therefore the total vertical distribution data needs to be calculated initially:

$$\theta_{\zeta,i} = \frac{\theta_{\zeta,\text{day},i} + \theta_{\zeta,\text{night},i}}{2} \quad (37)$$

Then the temperature scaling factor of each size class in a discretized water column can be obtained according to their vertical distribution:

$$k_{\zeta,i} = \theta_{\zeta,i} \cdot Q_{10}^{\frac{T_{\zeta} - T_{\text{ref}}}{10}} \quad (38)$$

where the water column temperature profile ranging from the surface (0 m) to the bottom, which can be obtained from observational data products or earth system model outputs. Finally, the scaling factor is integrated over the vertical distribution.

$$k_i = \sum_{\zeta} k_{\zeta,i} \quad (39)$$

k_i can be used in temperature effects on physiological rates Eq. 6, 7, 8. The implementation is hard-coded, and embedded in the function `setupVertical()`.

setupVertical2

To simplify the temperature input, three temperature inputs (T_p , T_m , T_b) are required rather than the water column temperature profile in `setupVertical2`. T_m represents the averaged mid-water temperature (500 - up to 1500 m). If T_m is not provided, $T_m = T_b$. The effective temperature of different size classes of each functional type are the averaged values of temperatures of their approximate vertical positions of day (T_{day}) and night (T_{night}).

$$T_e = \frac{T_{\text{day}} + T_{\text{night}}}{2} \quad (40)$$

Note this T_e is different from the one in `setupBasic` and `setupBasic2`. The T_e is taken into Eq. 34 as T , and updates the temperature-dependent physiological rates (Eq. 6, 7, 8). The implementation is hard-coded in the function `setupVertical2()`.

The temperatures associated with each size class and functional type follow the vertical distributions:

- Small pelagics always stay in the surface pelagic waters ($T_{\text{day}} = T_{\text{night}} = T_p$).
- Mesopelagics at night stay in the surface pelagic waters ($T_{\text{night}} = T_p$), whereas their vertical distribution varies with depth and photic conditions during the day:

$$T_{\text{day}} = \begin{cases} T_{\text{m}}, & z_{\text{dvm}} \neq z_{\text{bottom}} \text{ and } z_{\text{dvm}} \neq 0 \\ T_{\text{b}}, & z_{\text{dvm}} = z_{\text{bottom}} \\ T_{\text{p}}, & z_{\text{dvm}} = 0 \end{cases} \quad (41)$$

- The small and medium-sized classes of large pelagics are in the surface pelagic waters both day and night ($T_{\text{day}} = T_{\text{night}} = T_{\text{p}}$). The large-sized classes of large pelagic fish are in the surface pelagic waters at night ($T_{\text{night}} = T_{\text{p}}$) and they are split into two groups during the day: half of them are always in the surface pelagic zone, and the habitat of the other half depends on the water column conditions. Their daytime temperature is the average of these two environmental temperatures:

$$T_{\text{day}} = \begin{cases} \frac{T_{\text{m}}+T_{\text{p}}}{2}, & z_{\text{dvm}} \neq z_{\text{bottom}} \text{ and } z_{\text{dvm}} \neq 0 \\ \frac{T_{\text{b}}+T_{\text{p}}}{2}, & z_{\text{dvm}} = z_{\text{bottom}} \\ T_{\text{p}}, & z_{\text{dvm}} = 0 \end{cases} \quad (42)$$

- Midwater predators at daytime stay at the dvm depth, bottom, or surface pelagic zone, depending on the water column depth and photic conditions:

$$T_{\text{day}} = \begin{cases} T_{\text{m}}, & z_{\text{dvm}} \neq z_{\text{bottom}} \text{ and } z_{\text{dvm}} \neq 0 \\ T_{\text{b}}, & z_{\text{dvm}} = z_{\text{bottom}} \\ T_{\text{p}}, & z_{\text{dvm}} = 0 \end{cases} \quad (43)$$

At nighttime, small and medium midwater predators are in the surface pelagic waters ($T_{\text{night}} = T_{\text{p}}$), whereas the habitat of large midwater predators varies with water column conditions:

$$T_{\text{night}} = \begin{cases} T_{\text{m}}, & z_{\text{dvm}} \neq z_{\text{bottom}} \text{ and } z_{\text{dvm}} \neq 0 \\ T_{\text{b}}, & z_{\text{dvm}} = z_{\text{bottom}} \\ T_{\text{p}}, & z_{\text{dvm}} = 0 \end{cases} \quad (44)$$

- Small demersals are pelagic-living ($T_{\text{day}} = T_{\text{night}} = T_{\text{p}}$); medium demersals are benthic-living ($T_{\text{day}} = T_{\text{night}} = T_{\text{b}}$). Large demersal fish habitats vary according to the water column conditions.

$$T_{\text{day}} = \begin{cases} T_{\text{m}}, & (z_{\text{bottom}} - z_{\text{dvm}}) < 1500 \text{ and } z_{\text{bottom}} \geq z_{\text{photic}} \\ T_{\text{b}}, & (z_{\text{bottom}} - z_{\text{dvm}}) \geq 1500 \\ \frac{T_{\text{b}}+T_{\text{p}}}{2}, & z_{\text{bottom}} < z_{\text{photic}} \end{cases} \quad (45)$$

$$T_{\text{night}} = \begin{cases} T_{\text{b}}, & z_{\text{bottom}} \geq z_{\text{photic}} \\ \frac{T_{\text{b}}+T_{\text{p}}}{2}, & z_{\text{bottom}} < z_{\text{photic}} \end{cases} \quad (46)$$

Setup of the size spectrum grid

The fish size span of a functional type is logarithmically discretized into n continuous size bins. Each size bin shares the same ratio z between the upper boundary size and lower boundary size:

$$z = \exp\left(\frac{\ln(M) - \ln(M_0)}{n}\right) \quad (47)$$

where M_0 and M are the smallest and largest fish in a functional type (boundary size). There are $n + 1$ boundaries of n size bins ($m_{b,i}$), including all lower boundary sizes ($m_{l,i}$) and upper boundary sizes ($m_{u,i}$).

$$m_{b,i} = \exp(\ln(M_0) + (i - 1) \ln(z)), \quad i \in [1, n + 1] \quad (48)$$

$$m_{l,i} = m_{b,i}, \quad i \in [1, n] \quad (49)$$

$$m_{u,i} = m_{b,i+1}, \quad i \in [1, n] \quad (50)$$

The geometric mean size of each size class m_i is:

$$m_i = \exp(\ln(m_{l,i}) + 0.5(\ln(z))), \quad i \in [1, n] \quad (51)$$

which can be comprehensively used for calculations, for instance, physiological rates and size-based feeding preference.

The size spectrum generation is done by calling the function `paramAddGroup()`. Also, see the source code of `makeGrid()`.

Default parameters

Table 2: Main parameters of FEISTY

Symbol	Description	Value ^a				Unit
		B1	B2	V1	V2	
a_c	Intercept for mass-specific maximum consumption rate			20		$g^{-b_c} yr^{-1}$
b_c	Exponent for mass-specific maximum consumption rate			-0.25		/
a_e	Intercept for mass-specific clearance rate			70		$m^2 g^{-b_e-1} yr^{-1}$
b_e	Exponent for mass-specific clearance rate			-0.2		/
a_m	Intercept for mass-specific metabolism rate			$0.2 * a_c$		$g^{-b_m} yr^{-1}$
b_m	Exponent for mass-specific metabolism rate			-0.175		/
ϵ_α	Assimilation efficiency			0.7		/
ϵ_r	Reproduction efficiency			0.01		/
μ_b	Background mortality			0.1		yr^{-1}
F_{max}	Maximum fishing mortality rate			0^b		yr^{-1}
η_{mature}	half-maturation size coefficient	$/^c$		0.25		/
$\eta_{fishing}$	half-harvesting size coefficient	$/^d$		0.05		/
θ_A	Large fish preference on small pelagic fish ^e			0.5		/
θ_D	Large demersal fish preference on pelagic prey		0.75	/	/	/
ω_0	baseline range of the vertical distribution		/		10	/
τ	range-increasing factor of vertical distribution		/		10	/
T_{ref}	reference temperature			10		$^{\circ}C$
Q_{10}	Rate of change for every 10 $^{\circ}C$ increase for clearance rate and maximum consumption			1.88		/
Q_{10m}	Rate of change for every 10 $^{\circ}C$ increase for metabolism		2.35		1.88	/
β	Preferred predator:prey mass ratio	/		400		/
σ	Width of size preference for feeding	/		1.3		/
z_{shelf}	Continental shelf depth	/	/		250	m
z_{photic}	Photic zone depth	/	/		150	m
m_{medium}	boundary size of small/medium fish class			0.5		g
m_{large}	boundary size of medium/large fish class			250		g

^a B1: setupBasic1, B2: setupBasic2, V1: setupVertical1, V2: setupVertical2.

^b In B1, the fishing mortality can also be assigned manually [[@petrik2019bottom](#)] (Eq. 31 does not apply). $\mu_{f,i}$ are constant values: Small fish $0 yr^{-1}$, Medium fish $0.03 yr^{-1}$, Large fish $0.3 yr^{-1}$.

^c Maturity level ψ_i is assigned manually in default B1. Only the last stage of each functional type has a value of 0.5 others are 0.

^d Fishing mortality $\mu_{f,i}$ is 0 in default B1.

^e In B1, θ_A only poses on medium-sized classes of small pelagic fish (the second size class). In B2, θ_A works on all small pelagic fish. In V1 and V2, θ_A works on medium-sized classes of small pelagic and mesopelagic fish.

Demonstration

FESITY includes four setups which each specifies the available functional groups and their parameters:

- setupBasic creates a basic three-functional type setup as described in (Petrik et al. 2019).
- setupBasic2 creates the same three-functional type setup as setupBasic(), but it allows more size numbers in each functional type, size-based maturity, generalized size-based feeding preference, and size-based fishing mortality.

- `setupVertical` makes a basic five-functional type setup that includes vertical distribution of resources and fish (Denderen et al. 2021).
- `setupVertical2` is the same as `setupVertical` but different it allows more size numbers in each functional type, size-based maturity, generalized size-based feeding preference, size-based fishing mortality, and simpler temperature input.

Basic simulation and visualization

Here we demonstrate some examples of the basic usage of simulating FEISTY and visualization. Before a FEISTY simulation, first, we need to generate a full parameter set:

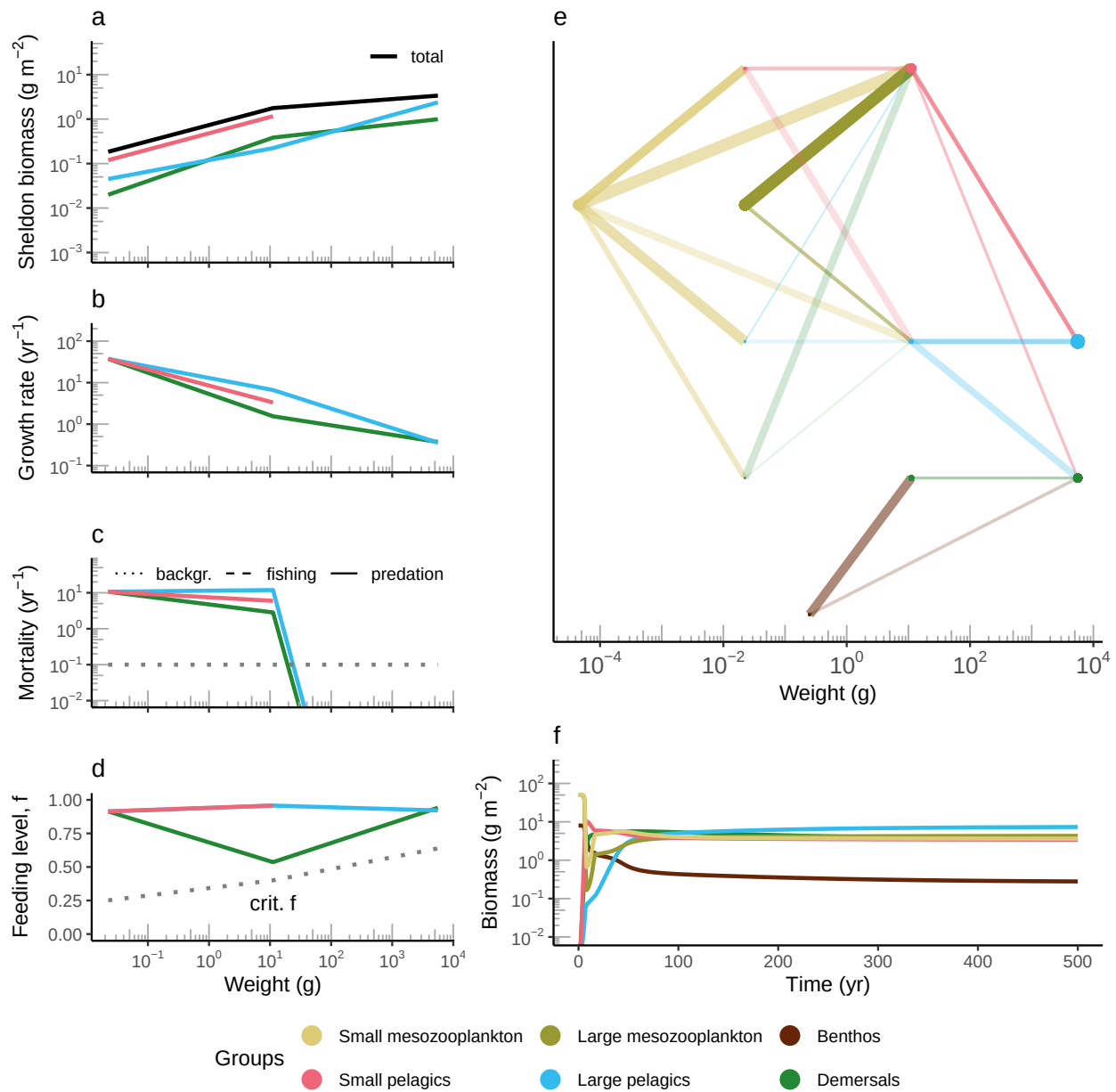
```
p <- setupBasic(szprod = 50, # small mesozooplankton production
               lzprod = 50, # large mesozooplankton production
               bprodin = 8, # benthos production
               depth = 150, # water column depth [m]
               Tp = 17, # pelagic layer averaged temperature [Celsius]
               Tb = 12) # sea floor temperature [Celsius]
```

This parameter set contains three functional types according to (Petrik et al. 2019), created by `setupBasic()`. Once the parameter set is ready, we can run the simulation by `simulateFEISTY()`:

```
sim <- simulateFEISTY(p=p, times=seq(0, 500, length.out=500), USEdll = T)
```

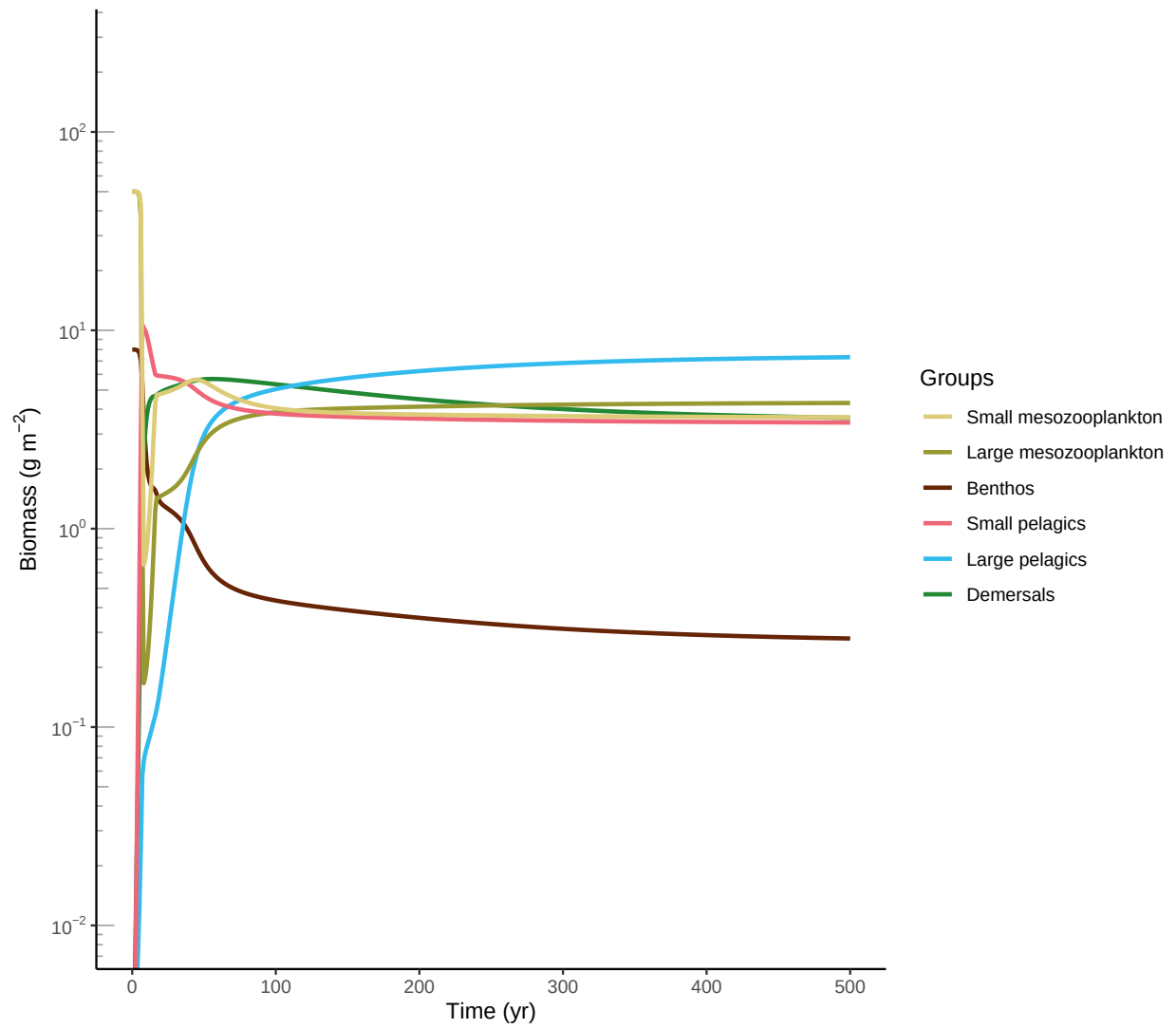
This simulation runs 500 years based on the parameter set `p` we defined above, and the time series output is in every year. `USEdll = T` denotes an almost identical parameter will be generated in the Fortran dll based on the arguments provided in `setupBasic()` and the core computation will be done by the Fortran dll and ultimately the results will be returned to R. Then we can visualize the simulation result. To get an overview, we can use `plotSimulation()` which gives information through a figure collection:

```
plotSimulation(sim)
```

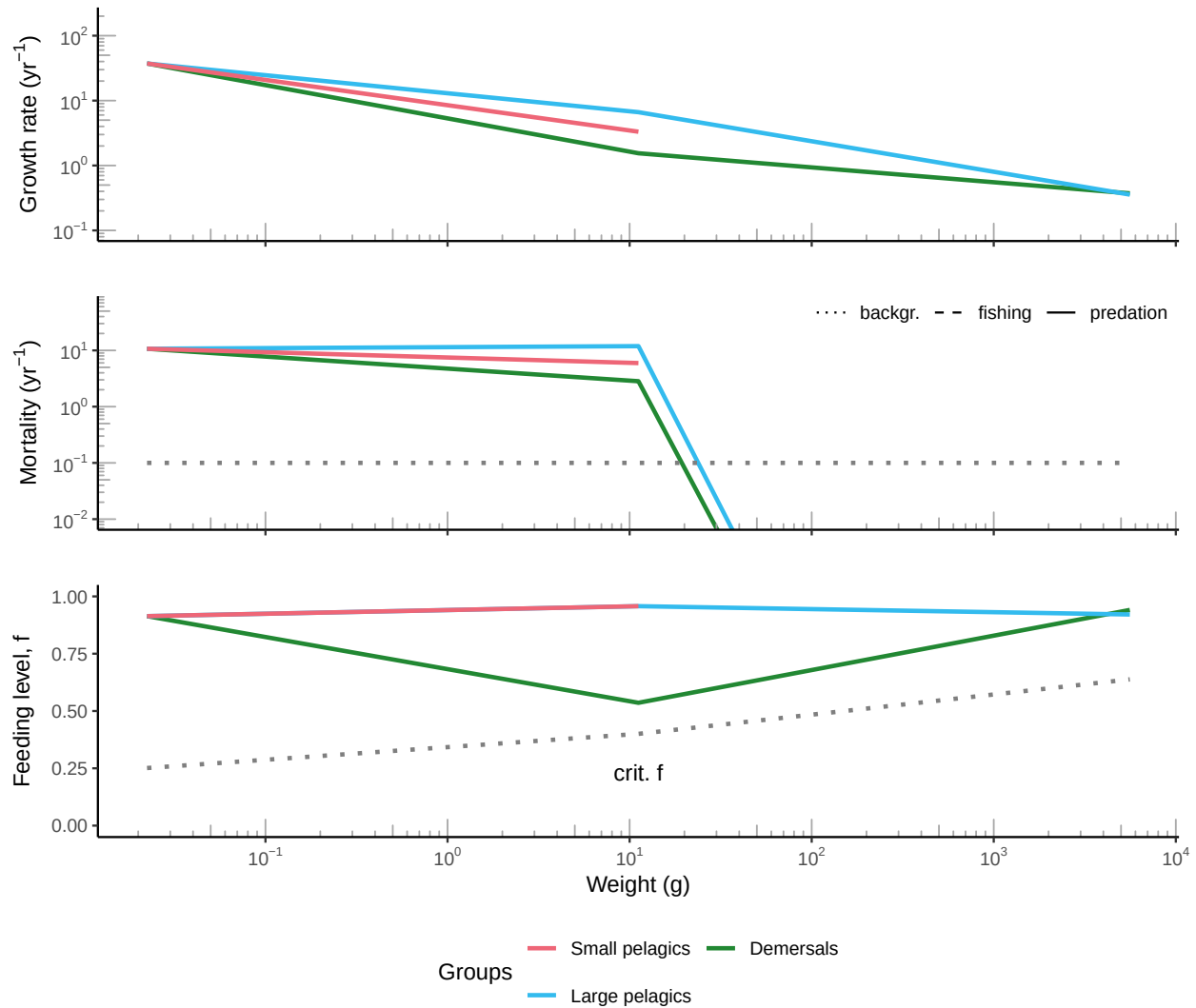


This figure collection includes `plotSpectra()`, `plotRates()`, `plotNetwork()`, and `plotBiomassTime()`. Each of these can be called independently. For instance:

```
plotBiomassTime(sim)
```



```
plotRates(sim)
```



Fishing mortality assignment

For simplicity, arguments of fishing in `setupbasic()` and `setupVertical()` only allow assigning fishing mortality to all functional types. Although the parameter set has been generated by calling setup functions e.g., `setupVertical()`, it is feasible to overwrite parameters manually. As a result, fishing mortality for a specific functional type can be assigned afterwards.

```
p=setupVertical2(szprod= 120,
                lzprod = 120,
                dfpho=200,
                depth = 700,
                nStages = 9,
                F=0) # no fishing for all size classes
names(p$mortF)=p$stagenames
df=data.frame(mortF_original=c(p$mortF[p$ix[[1]]],p$mortF[p$ix[[5]]]))
# assign 0.2/year as the maximum fishing mortality to small pelagic fish
p=setFishing(p=p,F=0.2,etaF=0.05,groupid=c(1))
# assign 0.3/year as the maximum fishing mortality to demersal fish
```

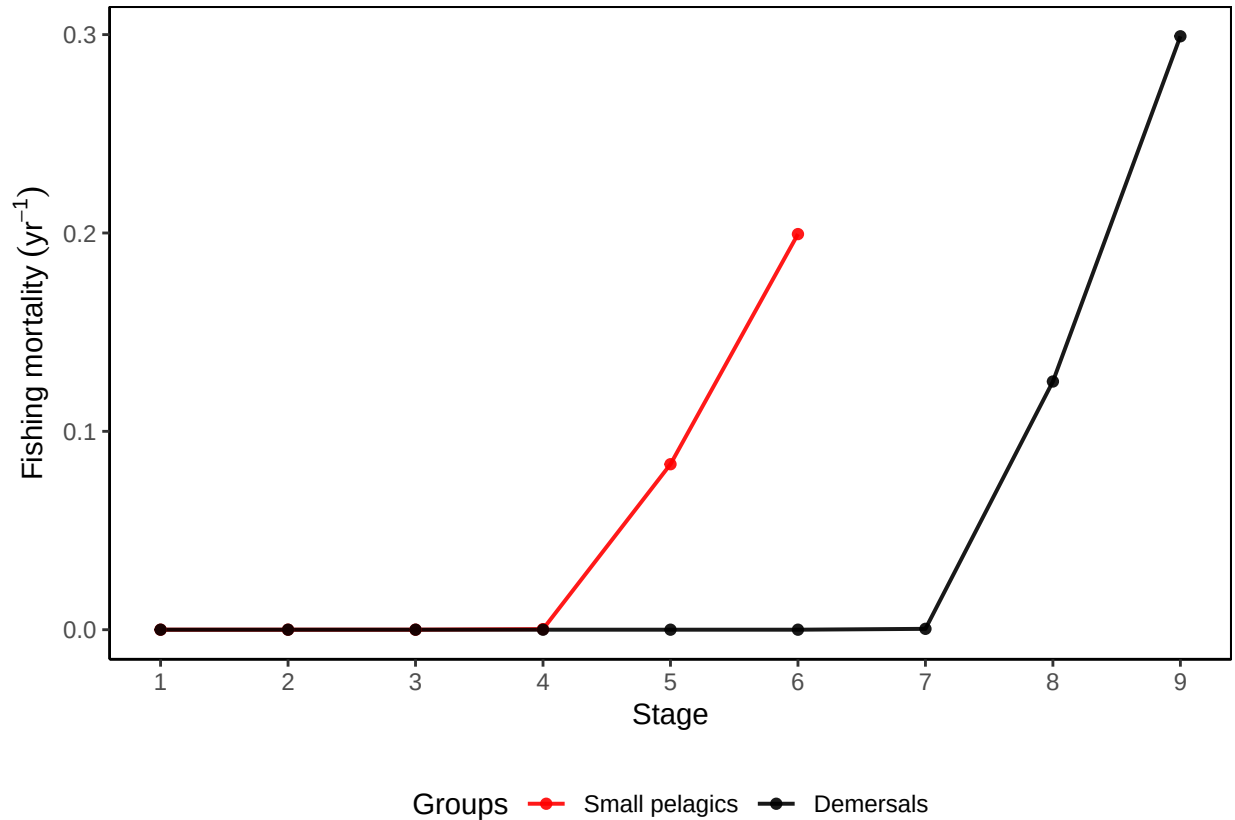
```
p=setFishing(p=p,F=0.3,etaF=0.05,groupid=c(5))
df=cbind(df,data.frame(mortF_new=c(p$mortF[p$ix[[1]]],p$mortF[p$ix[[5]]])))
knitr::kable(df,caption="Fishing mortality before and after assignment")
```

Table 3: Fishing mortality before and after assignment

	mortF_original	mortF_new
smallPel_1	0	0.0000000
smallPel_2	0	0.0000000
smallPel_3	0	0.0000006
smallPel_4	0	0.0002858
smallPel_5	0	0.0834188
smallPel_6	0	0.1994425
demersals_1	0	0.0000000
demersals_2	0	0.0000000
demersals_3	0	0.0000000
demersals_4	0	0.0000000
demersals_5	0	0.0000000
demersals_6	0	0.0000009
demersals_7	0	0.0004287
demersals_8	0	0.1251281
demersals_9	0	0.2991638

```
df=data.frame("Stage"=1:length(p$ix[[1]]), "mortF"=p$mortF[p$ix[[1]]],"Groups"="smallPel")
df=rbind(df,data.frame("Stage"=1:length(p$ix[[5]]),
                       "mortF"=p$mortF[p$ix[[5]]],"Groups"="demersals"))
df$Groups=factor(df$Groups,levels=c("smallPel","demersals"))
# plot of fishing mortality of small pelagics and demersals
fig=ggplot(df, aes(x = Stage, y = mortF, color = Groups))+
  geom_line(linewidth = 0.7,alpha=0.9)+
  geom_point(size=1.5,alpha=0.9)+
  labs(x = expression("Stage"), y = expression("Fishing mortality"~(yr^{-1}))) +
  scale_color_manual(values = c("red", "black"),labels=c("Small pelagics","Demersals")) +
  scale_x_continuous(breaks = unique(df$Stage))+
  theme(panel.background = element_rect(fill = "white"),
        panel.border = element_rect(color = "black", fill = NA),
        axis.line = element_line(color = "black"),
        #legend.title = element_blank(),
        legend.key = element_rect(fill = "transparent", color = "transparent"),
        legend.position = "bottom")

fig
```

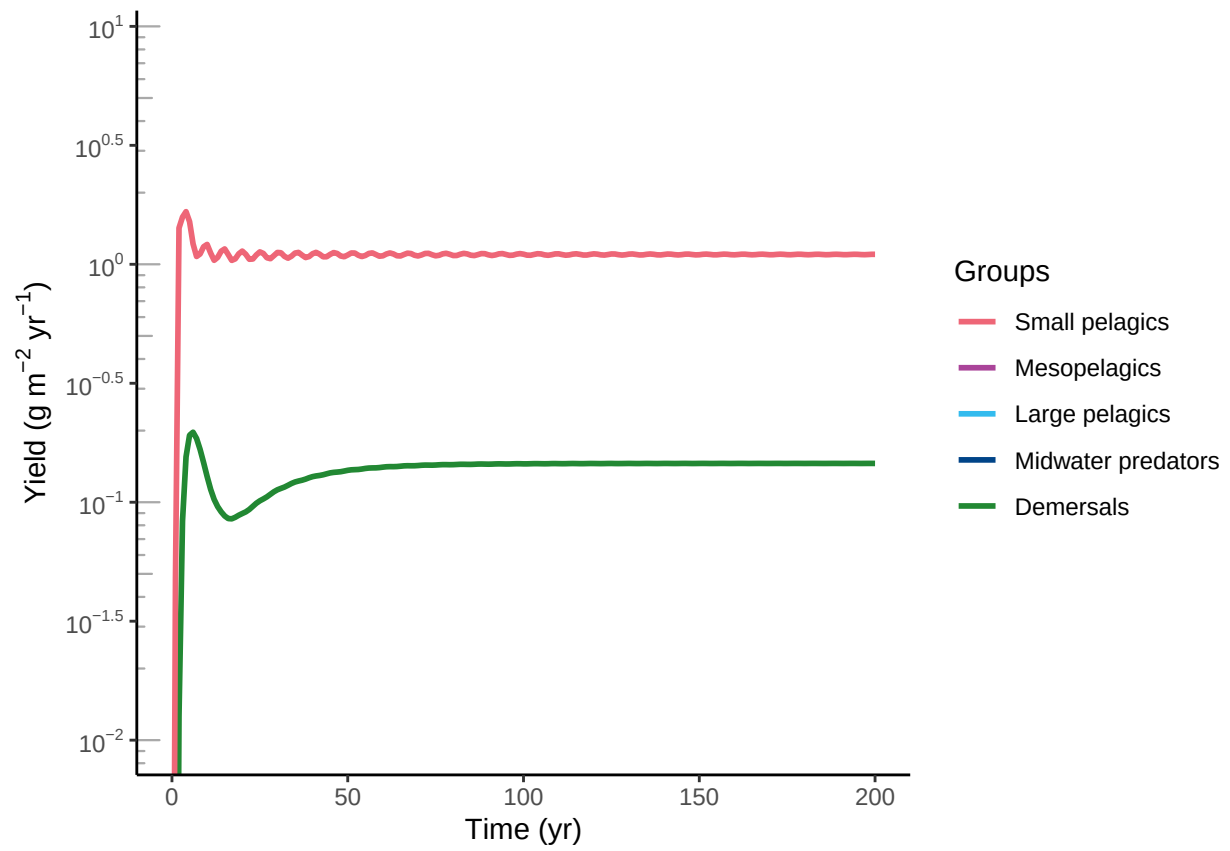


Since fishing mortality was assigned manually to two of three fish functional types, which is not the ready-to-use function `setupXXX()` can do, we must turn on `USEdll` and `bCust` flags for simulations. It means we want to run the simulation based on a parameter set we customized. All core parameters for simulation will be transmitted from R to Fortran dll rather than generated in Fortran. These two flags are TRUE as default. If `USEdll = F`, it means the simulation is done in R (slower but helpful when debugging).

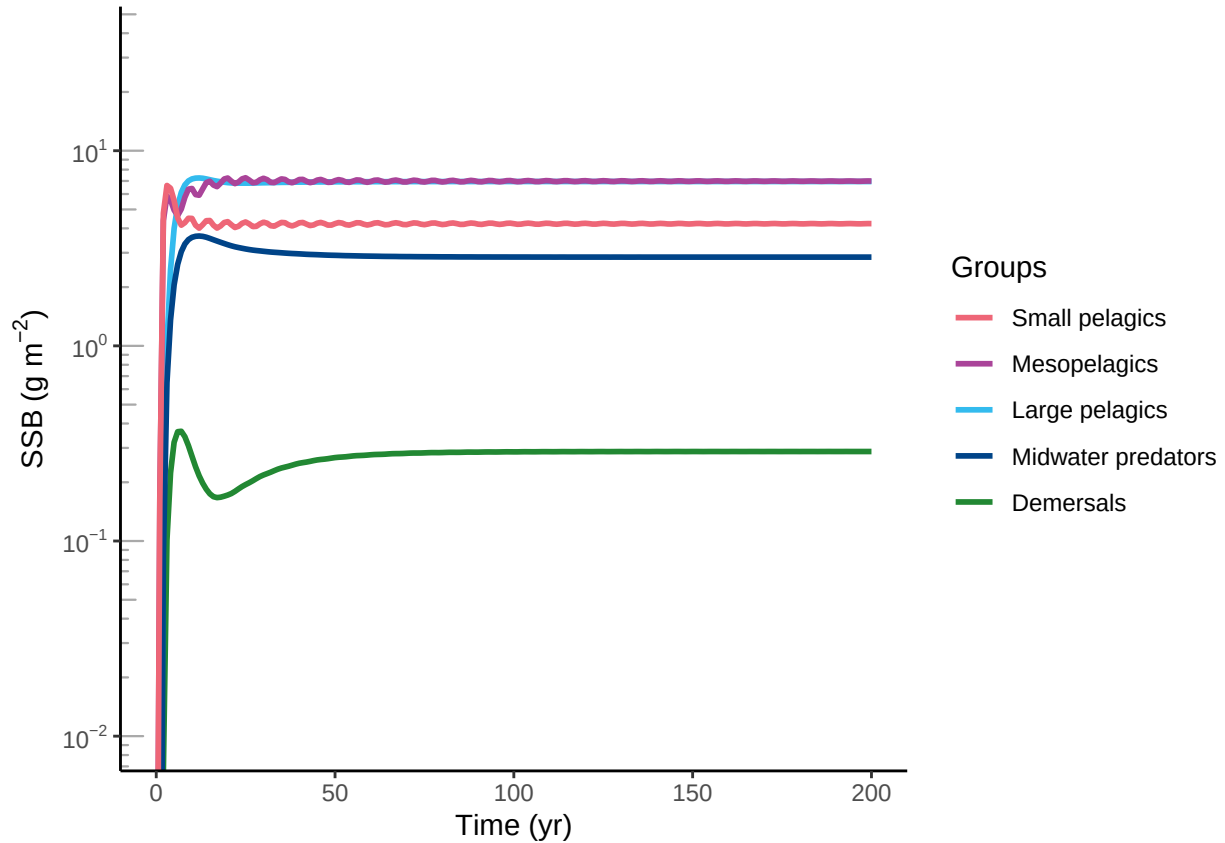
```
sim=simulateFEISTY(p = p, tEnd = 200, USEdll = T, bCust = T)
```

The following functions can be used for visualizing yield and spawning stock biomass changes over time.

```
plotYieldtime(sim)
```



```
plotSSBtime(sim)
```



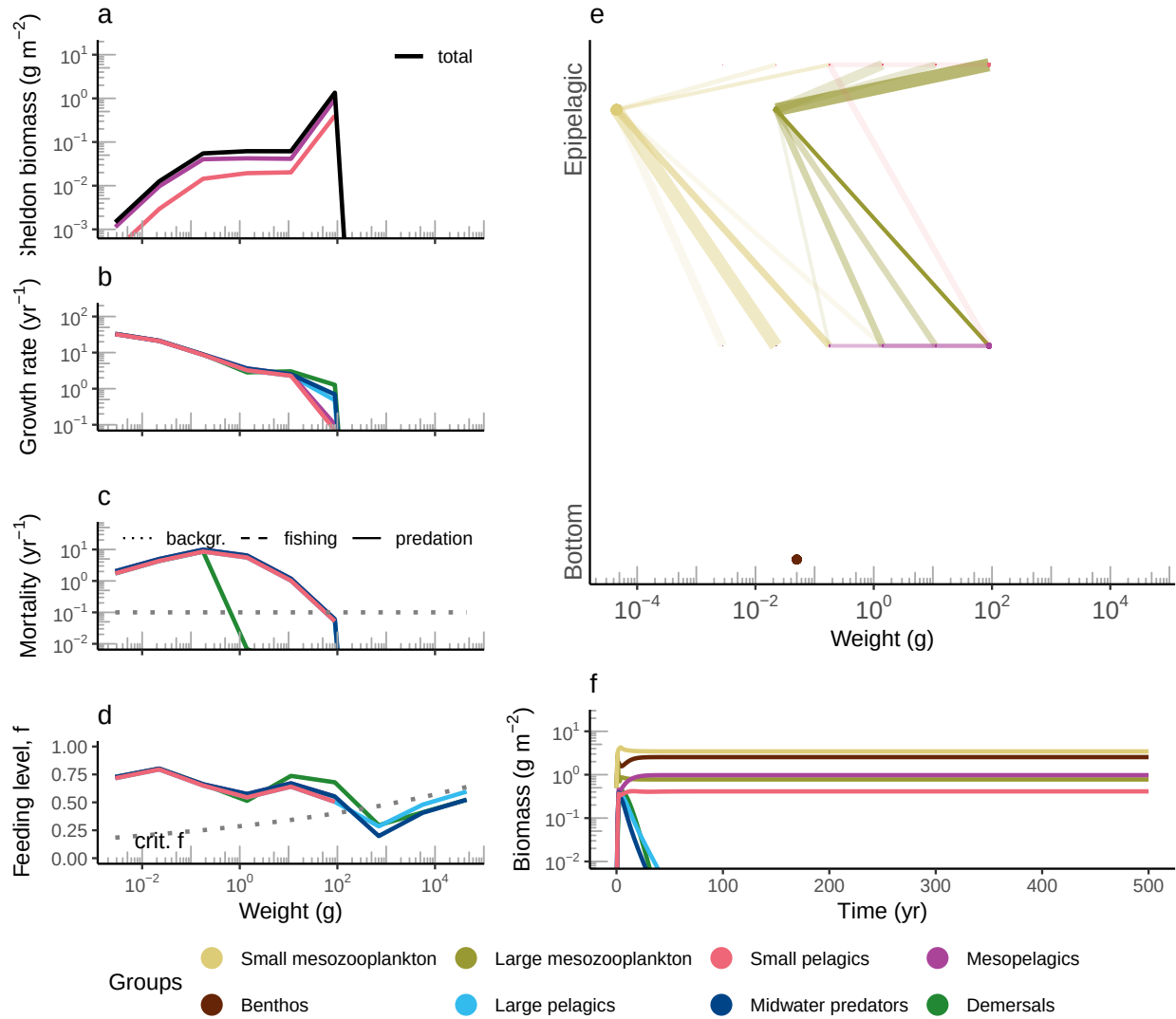
Bottom-up control examples

One of the goals of the FEISTY model is to obtain emergent fish food webs that vary with environmental conditions (i.e. bottom-up mechanisms that control fish communities). Here we demonstrate two model runs with one describing an oligotrophic system and the other one a more eutrophic system based on `setupVertical2`.

```
p1=setupVertical2(depth=1000,szprod=5, lzprod=5,dfpho = 130) # oligotrophic 1000 meter
p2=setupVertical2(depth=1000,szprod=100, lzprod=100,dfpho =380) # eutrophic 1000 meter
```

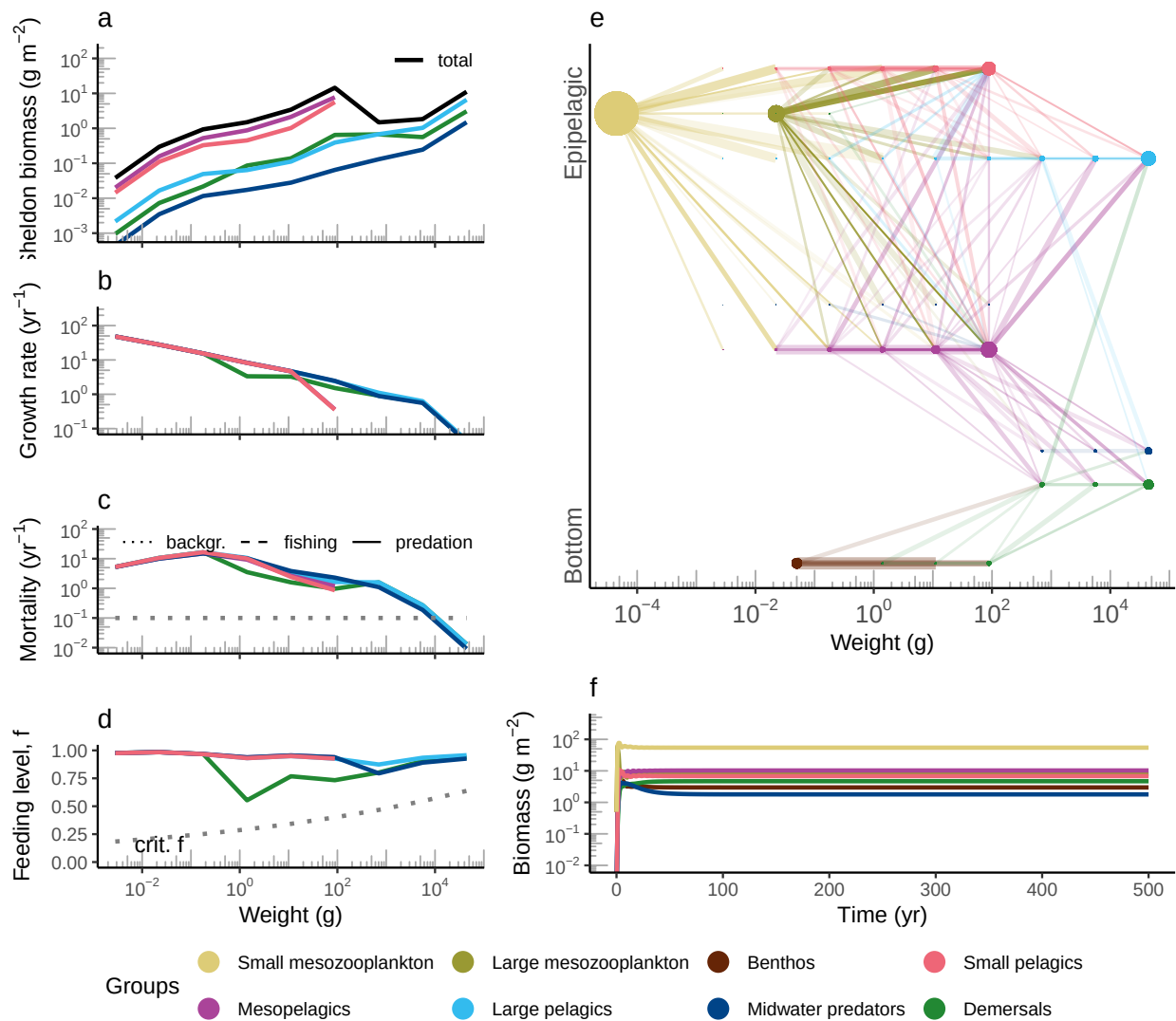
In the oligotrophic water, large pelagic fish, midwater predators, and demersal fish cannot survive (panel f), since the resource productions are low and do not provide enough food for them (panel d).

```
sim1=simulateFEISTY(p=p1,tEnd=500)
plotSimulation(sim1)
```



On the contrary, in the eutrophic water, high resource productions support the existence of all five functional types (panel f). All fish have feeding levels that are higher than their critical feeding levels (panel d).

```
sim2=simulateFEISTY(p=p2,tEnd=500)
plotSimulation(sim2)
```



Time-series simulation

The version 1.1 of the FEISTY package provides functions allowing the FEISTY model to be able to couple with datasets or other model products that provide time-series inputs, of temperature, zooplankton biomass, zooplankton production, benthic production, as well as fishing mortality. Details of how to set up and run time-series simulations can be found on the help page of the `setupTimeseries()` function. There are revisions to the original FEISTY model.

Resources dynamics

Zooplankton

Zooplankton dynamics in the original FEISTY model are revoked as the zooplankton biomass are time-varying input.

Benthos

Due to the limited access to benthos biomass data, the benthos community follows the logistic growth according to the time-varying production data, which is limited by the carrying capacity (K_i , set by users):

$$\frac{dR_i}{dt} = r_i \cdot \left(1 - \frac{R_i}{K_i}\right) - \mu_{p,i} \cdot R_i. \quad (52)$$

Note, this is slightly different from Eq. 5. The r_i in Eq. 5 is the growth rate in the unit of per time, while the r_i here represents the production (biomass/area/time).

Down-regulation of zooplankton consumption

To avoid the creation of new mass in the model, the consumption by the fish predators must not exceed the corresponding zooplankton production ($P_{R,j}$, in mass/area/time). If the total consumption by all fish, as calculated by the product of predation mortality on zooplankton and zooplankton biomass ($\mu_{p,j}R_j$, in mass/area/time) exceeds the resource production, the consumption of the resource has to be down-regulated.

When consumption of one food item (here the resource) is decreased the predator can compensate by eating more of other prey. For simplicity, however, we assume that only the resource consumption is decreased, and the consumption of other prey is not increased to compensate. To follow this assumption, we introduce a down-regulation factor $f_{dr,j}$, and henceforth several variable changes. The total consumption of all predators of the resource prey j is equal to the predation mortality loss of that prey (Eq. 18). That predation mortality is updated to:

$$\mu_{p,j} = f_{dr,j} \sum_i \frac{V_i \theta_{i,j}}{E_i + C_{\max,i}} C_{\max,i} B_i, \quad (53)$$

where E_i is the total encountered food. j denotes indices of prey including all resource and fish. The down-regulation factor $f_{dr,j}$ is the ratio of the resource production and resource predation loss:

$$f_{dr,j} = \begin{cases} \max\{1, \frac{P_{R,j}}{\mu_{p,j}R_j}\}, & j \in \text{resource requiring down-regulation} \\ 1, & j \in \text{other prey} \end{cases} \quad (54)$$

For the other prey whose predation loss do not need down-regulation, the factor is 1 to keep everything unchanged.

The predation loss of a resource prey reflects the consumption of its predators, which requires corresponding updates to maintain mass conservation within the system. We begin by revising the feeding levels of predators. The mass-specific feeding level of a predator f_i (Eq. 10, also see Eq. 11) is consequently modified to

$$f_i = \frac{V_i}{E_i + C_{\max,i}} \sum_j f_{dr,j} \theta_{i,j} B_j. \quad (55)$$

This procedure ensures that mass is not created in the system. However, it does not ensure perfect mass balance. If the fish predators do not eat all the zooplankton production, there is an “excess” zooplankton production which is not accounted for. Further, as mentioned above, the fish are not allowed to compensate for the reduction in zooplankton consumption by an increase in consumption of other prey. We believe that this latter assumption has a quite small impact.

Here, we provide an example demonstrating the down-regulation of large mesozooplankton consumption in a time-series simulation. When the zooplankton consumption by fish hits the zooplankton production (green lines above red), it is down-regulated (blue lines overlapped with red). The consumption value after down-regulation will be used in derivative calculation in the specific time steps.

```

# load example data from package
data(tsinput_example_1850_2014)
# setup time series based on setupBasic2
# 10-year time-series simulation after 2-year spinning up (loop 4 times)
p = setupTimeseries(p = setupBasic2(depth = depth, nStages = 9),
                    tStep_ts = 1/12,
                    tSpin = 2,
                    nSpinloop = 4,
                    Tp_ts = Tp[1:120],
                    Tm_ts = Tm[1:120],
                    Tb_ts = Tb[1:120],
                    szbio = Zbio[1:120]/2,
                    lzbio = Zbio[1:120]/2,
                    szprod_ts = Zhploss[1:120]/2,
                    lzprod_ts = Zhploss[1:120]/2,
                    dfbot_ts = dfbot[1:120],
                    Fsmpt_ts = Fspel[1:120],
                    Flgp_ts = Flpel[1:120],
                    Fdem_ts = Fdem[1:120])

```

```

## Time-series input: szbio_ts, lzbio_ts, szprod_ts, lzprod_ts, dfbot_ts, Tp_ts, Tm_ts, Tb_ts, Fsmpt_ts,
## Time-series length: 10 years.
## Time-series time step: 1/12 years.
## Spin-up length: 2 years.
## Spin-up loop number: 4.

```

```

# Run simulation by Fortran
simF = simulateFEISTY(p = p, USEdll = T)

```

```

## spin-up progress: 25.00%
## spin-up progress: 50.00%
## spin-up progress: 75.00%
## spin-up progress: 100.00%

```

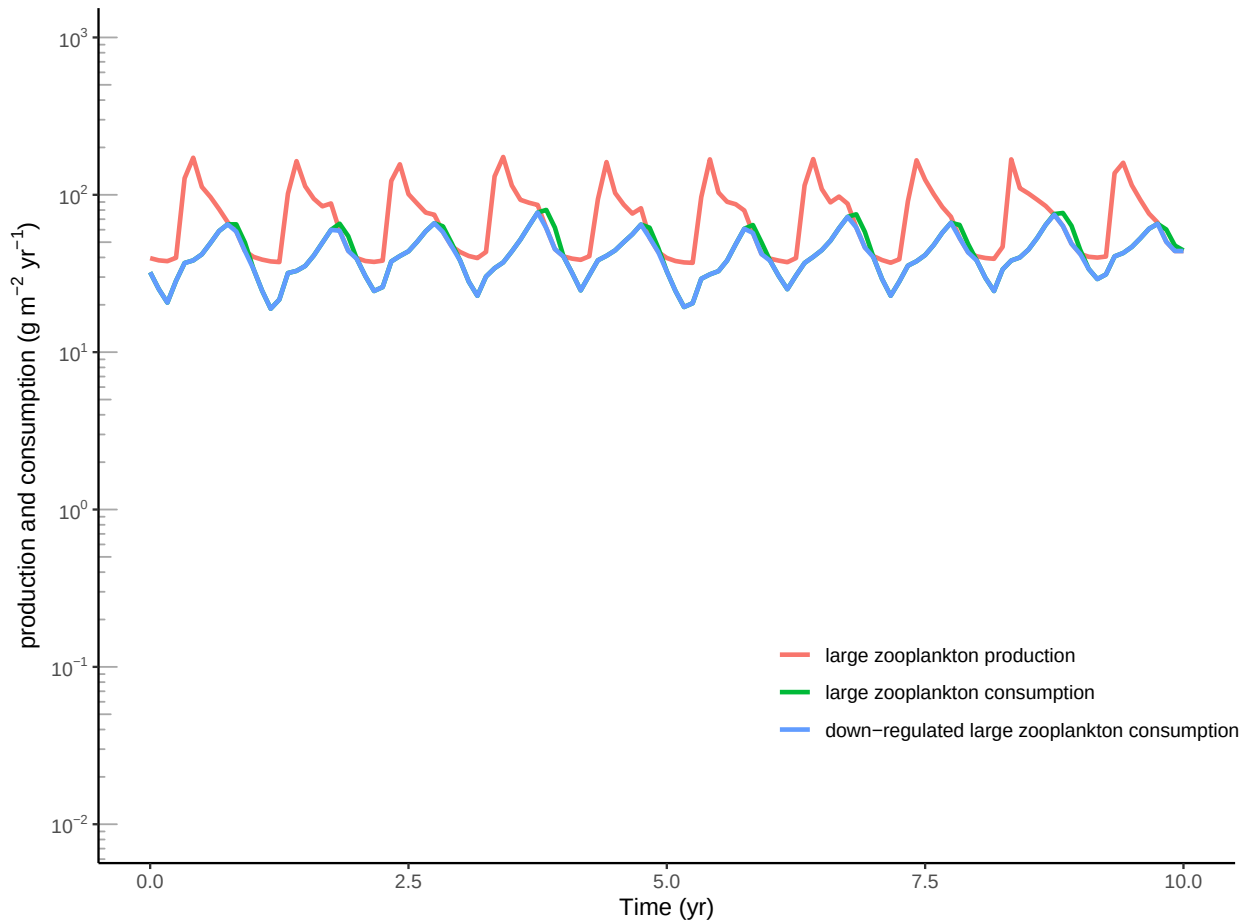
```

# Set dataframe for plotting
diagts <- data.frame(group = factor(rep(c("large zooplankton production",
                                         "large zooplankton consumption",
                                         "down-regulated large zooplankton consumption"),
                                         each = length(simF$t)),
                      levels = c("large zooplankton production",
                                  "large zooplankton consumption",
                                  "down-regulated large zooplankton consumption")),
                    val = c(as.numeric(p$lzprod_ts),
                           as.numeric(simF$lgzcsp),
                           as.numeric(simF$lgzcsp_dr)),
                    t = rep(simF$t,3))

# Plot
ggplot(data = diagts, aes(x = t, y = val, color = group, group = group)) +
  geom_line(linewidth = 1) + # Plot lines
  xlab("Time (yr)") +
  ylab(expression("production and consumption (g m-2* yr-1*)")) +
  annotation_logticks(sides = "l", linewidth = 0.4, colour = "darkgrey") +

```

```
coord_cartesian(ylim = c(1E-2, max(1E-2 * 100, max(diagts$val) * 5))) +
scale_y_log10(breaks = trans_breaks("log10", function(x) 10^x),
             labels = trans_format("log10", math_format(10^.x))) +
theme_classic()+
theme(legend.key = element_blank())+
theme(legend.position = "inside",
      legend.position.inside = c(0.8, 0.2))+
labs(color = NULL)
```



Discrepancies between the simulation results produced by R and Fortran

The time-series simulation results produced by R and Fortran could be different due to the nature of chaotic systems, which are highly sensitive to even the tiniest perturbations. These perturbations may be introduced from the ode solving in R and Fortran. Here, two cases are presented. In the first case, the results from R and Fortran are nearly identical. In the second case, the discrepancies start to emerge after around 80 years.

This “drifting apart” of the solutions is not an error but is due to the sometimes chaotic nature of the solution of the ODE system. The solutions are not always chaotic (for instance, the first case). The fact that the solutions may be chaotic shows the limitation of what we can expect the model to predict: we can predict average values, but not the accurate time trajectory of any of the species groups.

```

# load example data from package
data(tsinput_example_1850_2014)
# setup time series based on setupVertical2 (9 stages)
# 165-year time-series simulation after 16.5-year spinning up (loop 4 times)
p = setupTimeseries(p = setupVertical2(photic = photic, depth = depth, nStages = 9),
                    tStep_ts = 1/12,
                    tSpin = 16.5,
                    nSpinloop = 4,
                    Tp_ts = Tp,
                    Tm_ts = Tm,
                    Tb_ts = Tb,
                    szbio = Zbio/2,
                    lzbio = Zbio/2,
                    szprod_ts = Zhploss/2,
                    lzprod_ts = Zhploss/2,
                    dfbot_ts = dfbot,
                    Fsmpt_ts = Fspel,
                    Fmesop_ts = Fmeso,
                    Flgp_ts = Flpel,
                    Fmidwp_ts = FmidP,
                    Fdem_ts = Fdem)

```

```

## Time-series input: szbio_ts, lzbio_ts, szprod_ts, lzprod_ts, dfbot_ts, Tp_ts, Tm_ts, Tb_ts, Fsmpt_ts,
## Time-series length: 165 years.
## Time-series time step: 1/12 years.
## Spin-up length: 33/2 years.
## Spin-up loop number: 4.

```

```

# Run simulation by Fortran
simF = simulateFEISTY(p = p, USEdll = T)

```

```

## spin-up progress: 25.00%
## spin-up progress: 50.00%
## spin-up progress: 75.00%
## spin-up progress: 100.00%

```

```

simR = simulateFEISTY(p = p, USEdll = F)

```

```

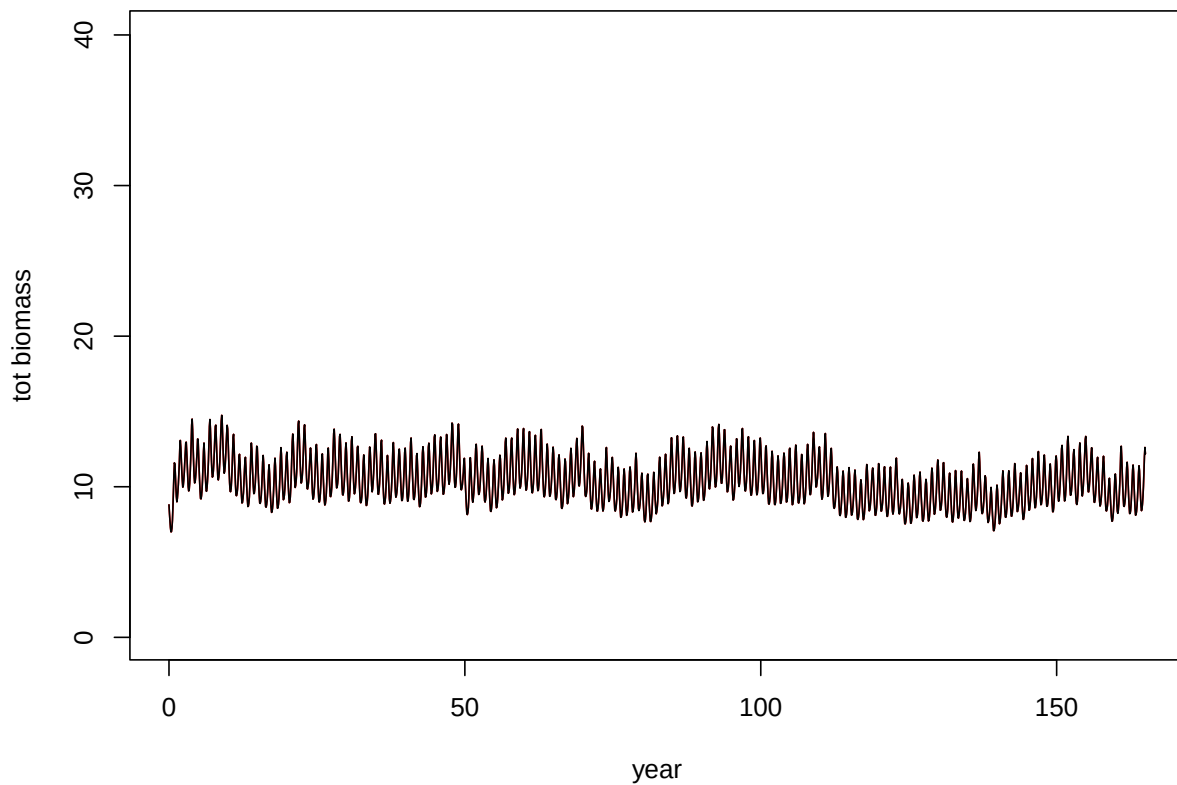
## spin-up progress: 25.00%
## spin-up progress: 50.00%
## spin-up progress: 75.00%
## spin-up progress: 100.00%

```

```

# Plot
plot(simR$t, rowSums(simR$totBiomass,2), type='l', xlab="year", ylab="tot biomass",
     log="", ylim=c(0.1,40), xlim=c(0,165), col='red')
lines(simF$t, rowSums(simF$totBiomass,2), type='l', col='black')

```



```
# setup time series based on setupVertical2 (3 stages)
# 165-year time-series simulation after 16.5-year spinning up (loop 4 times)
p = setupTimeseries(p = setupVertical2(photic = photic, depth = depth, nStages = 3),
                    tStep_ts = 1/12,
                    tSpin = 16.5,
                    nSpinloop = 4,
                    Tp_ts = Tp,
                    Tm_ts = Tm,
                    Tb_ts = Tb,
                    szbio = Zbio/2,
                    lzbio = Zbio/2,
                    szprod_ts = Zhploss/2,
                    lzprod_ts = Zhploss/2,
                    dfbot_ts = dfbot,
                    Fsmp_ts = Fspel,
                    Fmesop_ts = Fmeso,
                    Flgp_ts = Flpel,
                    Fmidwp_ts = FmidP,
                    Fdem_ts = Fdem)
```

```
## Time-series input: szbio_ts, lzbio_ts, szprod_ts, lzprod_ts, dfbot_ts, Tp_ts, Tm_ts, Tb_ts, Fsmp_ts,
## Time-series length: 165 years.
## Time-series time step: 1/12 years.
```

```
## Spin-up length: 33/2 years.  
## Spin-up loop number: 4.
```

```
# Run simulation by Fortran
```

```
simF = simulateFEISTY(p = p, USEd11 = T)
```

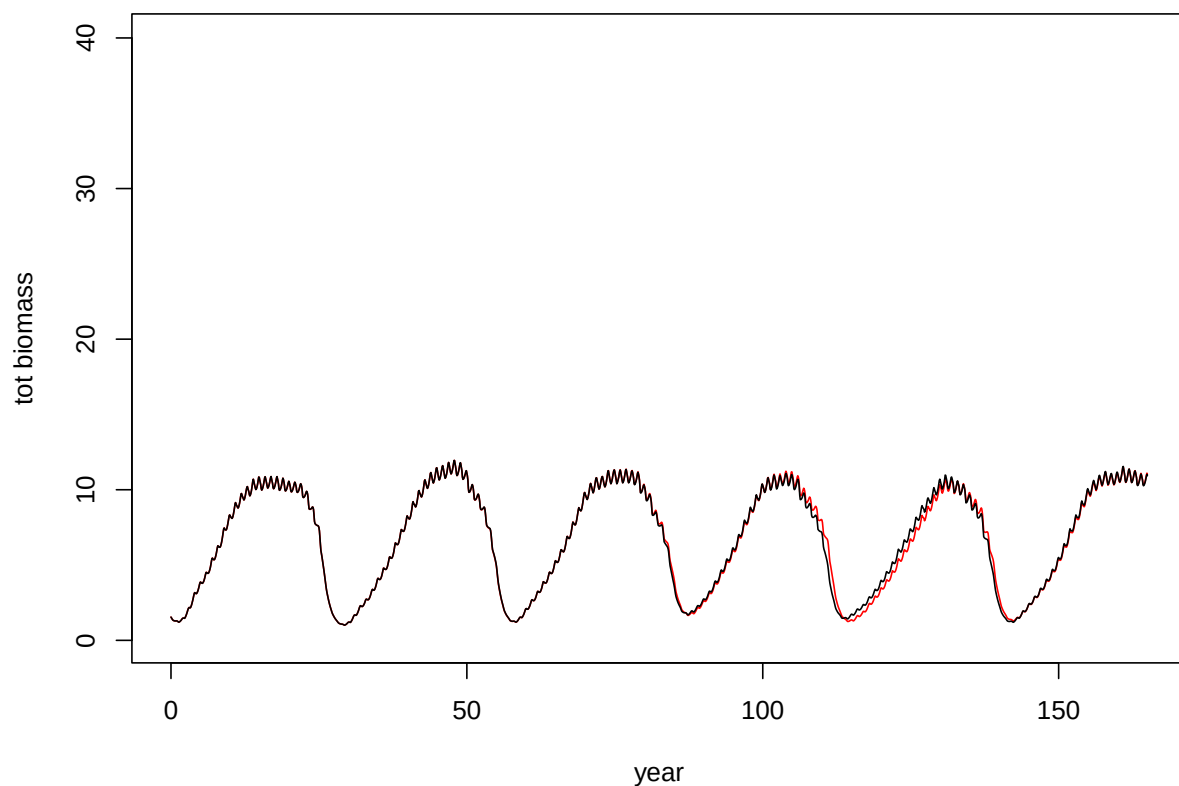
```
## spin-up progress: 25.00%  
## spin-up progress: 50.00%  
## spin-up progress: 75.00%  
## spin-up progress: 100.00%
```

```
simR = simulateFEISTY(p = p, USEd11 = F)
```

```
## spin-up progress: 25.00%  
## spin-up progress: 50.00%  
## spin-up progress: 75.00%  
## spin-up progress: 100.00%
```

```
# Plot
```

```
plot(simR$t, rowSums(simR$totBiomass,2), type='l', xlab="year", ylab="tot biomass",  
      log="", ylim=c(0.1,40), xlim=c(0,165), col='red')  
lines(simF$t, rowSums(simF$totBiomass,2), type='l', col='black')
```



Based on the second case (Fortran), an additional perturbation is introduced while other settings remain the same. Two simulation results are visualized, demonstrating the discrepancies.

```
# load example data from package
data(tsinput_example_1850_2014)
# add a perturbation
Zbio[100]
```

```
## [1] 6.178579
```

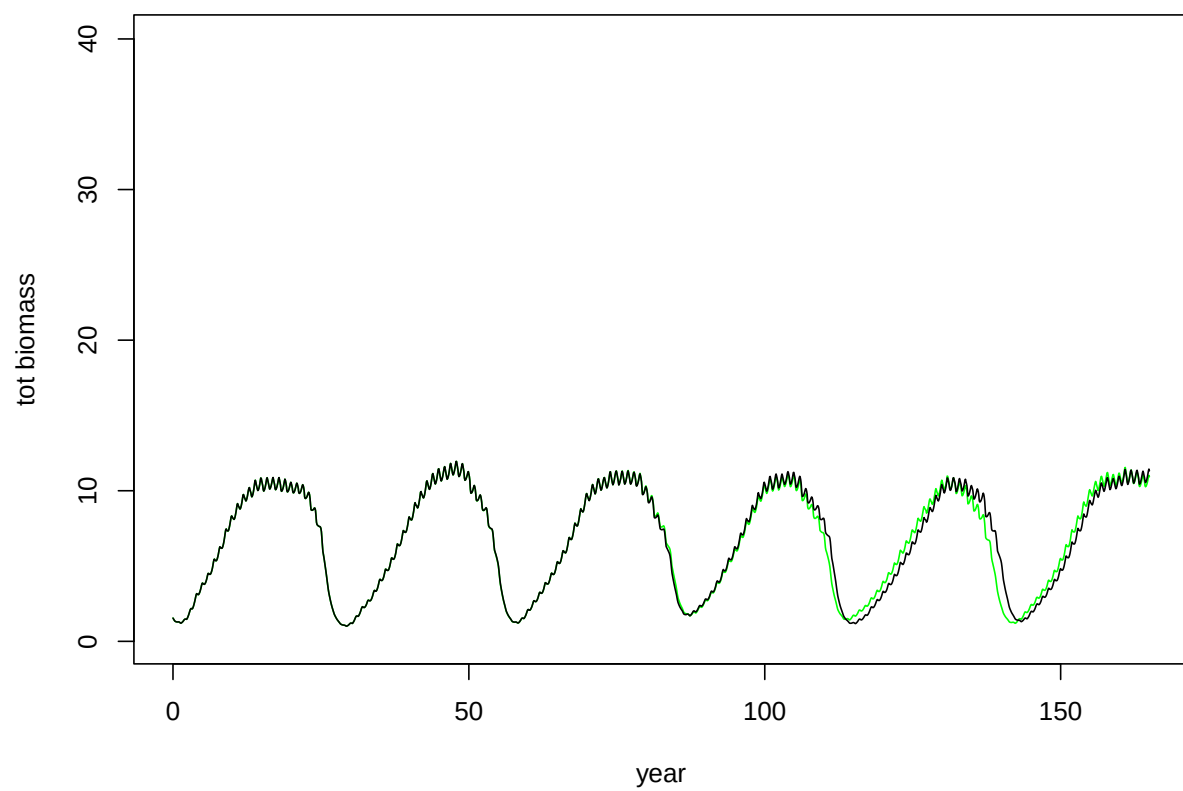
```
Zbio[100]=Zbio[100]+1E-5
# setup time series based on setupVertical2 (3 stages)
# 165-year time-series simulation after 16.5-year spinning up (loop 4 times)
p = setupTimeseries(p = setupVertical2(photic = photic, depth = depth, nStages = 3),
                    tStep_ts = 1/12,
                    tSpin = 16.5,
                    nSpinloop = 4,
                    Tp_ts = Tp,
                    Tm_ts = Tm,
                    Tb_ts = Tb,
                    szbio = Zbio/2,
                    lzbio = Zbio/2,
                    szprod_ts = Zhploss/2,
                    lzprod_ts = Zhploss/2,
                    dfbot_ts = dfbot,
                    Fsmpt_ts = Fspel,
                    Fmesop_ts = Fmeso,
                    Flgp_ts = Flpel,
                    Fmidwp_ts = FmidP,
                    Fdem_ts = Fdem)
```

```
## Time-series input: szbio_ts, lzbio_ts, szprod_ts, lzprod_ts, dfbot_ts, Tp_ts, Tm_ts, Tb_ts, Fsmpt_ts,
## Time-series length: 165 years.
## Time-series time step: 1/12 years.
## Spin-up length: 33/2 years.
## Spin-up loop number: 4.
```

```
# Run simulation by Fortran
simF2 = simulateFEISTY(p = p, USEd11 = T)
```

```
## spin-up progress: 25.00%
## spin-up progress: 50.00%
## spin-up progress: 75.00%
## spin-up progress: 100.00%
```

```
# Plot
plot(simF$t, rowSums(simF$totBiomass,2),type='l', xlab="year",ylab="tot biomass",
     log="", ylim=c(0.1,40), xlim=c(0,165), col='green')
lines(simF2$t, rowSums(simF2$totBiomass,2), type='l', col='black')
```



References

- Andersen, Ken H. 2019. *Fish Ecology, Evolution, and Exploitation: A New Theoretical Synthesis*. Princeton University Press.
- De Roos, André M, Tim Schellekens, Tobias Van Kooten, Karen Van De Wolfshaar, David Claessen, and Lennart Persson. 2008. “Simplifying a Physiologically Structured Population Model to a Stage-Structured Biomass Model.” *Theoretical Population Biology* 73 (1): 47–62.
- Denderen, P Daniël van, Colleen M Petrik, Charles A Stock, and Ken H Andersen. 2021. “Emergent Global Biogeography of Marine Fish Food Webs.” *Global Ecology and Biogeography* 30 (9): 1822–34.
- Petrik, Colleen M, Charles A Stock, Ken H Andersen, P Daniël van Denderen, and James R Watson. 2019. “Bottom-up Drivers of Global Patterns of Demersal, Forage, and Pelagic Fishes.” *Progress in Oceanography* 176: 102124.